

SmartFarm

Rapport de projet

Projet de Génie Logiciel - Delaunay / Voronoï

ING1 - Génie Informatique - Groupe GI5

Équipe F



LEMENAND Tom, LIGANOV Illya, LUIGGI Oscar, MORILLE Paul, MOHAMED Jilani

Tutrice de projet : Eva Ansermin
Année 2025 - 2026 - CY Tech

Sommaire

1. Introduction	3
2. La naissance du projet	3
3. Conception et choix techniques.....	4
4. Organisation de l'équipe	4
5. Réalisation technique.....	5
5.1 Le modèle de données.....	5
5.2 Les algorithmes de Delaunay et Voronoï.....	6
5.3 L'interface graphique JavaFX	7
5.4 Aperçu de l'interface.....	7
5.5 La version en ligne de commande	9
5.6 La sauvegarde et le chargement.....	9
6. Difficultés rencontrées.....	9
7. Les diagrammes UML.....	10
8. Conclusion	10

1. Introduction

Ce rapport présente le travail que nous avons réalisé dans le cadre du Projet de Génie Logiciel de première année. Notre équipe est composée de cinq étudiants : Tom Lemenand, Illya Liganov, Oscar Luiggi, Paul Morille et Jilani Mohamed. Le projet que nous avons choisi consiste à manipuler un partitionnement de l'espace en deux dimensions à l'aide d'un diagramme de Voronoï et d'une triangulation de Delaunay, et de pouvoir interagir avec cette carte de manière visuelle.

Le sujet imposait un cœur technique précis (les deux algorithmes cités au dessus), mais nous laissait libre sur la problématique concrète à traiter. On a choisi de partir sur une application liée à l'agriculture, qu'on a appelée SmartFarm, et dont on explique l'origine un peu plus loin.

Dans ce document on revient d'abord sur la naissance du projet et sur le changement de direction qu'on a dû opérer au début. On présente ensuite nos choix de conception, la façon dont l'équipe s'est organisée, puis la réalisation technique elle-même (le modèle, les algorithmes, les deux interfaces). On termine par les difficultés qu'on a rencontrées et un petit bilan. Les diagrammes UML, qui sont assez grands, sont placés en annexe.

2. La naissance du projet

SmartFarm n'est pas parti de zéro. Le nom et l'idée venaient déjà d'un travail qu'on avait mené dans les modules d'Éthique des sciences et de Design. À ce moment là, on avait imaginé un outil d'agriculture de précision basé sur des capteurs : des capteurs posés dans les champs auraient mesuré l'humidité du sol et l'ensoleillement, pour arroser automatiquement et de façon plus fine et intelligente les zones qui en avaient besoin. Le cahier des charges du projet demandait justement de réutiliser autant que possible ce qu'on avait déjà fait, donc on est repartis logiquement sur SmartFarm.

Le problème est apparu quand le sujet technique a été dévoilé. Avec un système de capteurs, on ne voyait pas vraiment comment utiliser Voronoï et Delaunay : ces algorithmes ont besoin de points de référence pour découper l'espace, et nos capteurs ne jouaient pas ce rôle. On s'est rendu compte assez vite qu'on était un peu à côté du sujet, et qu'il fallait revoir notre idée pour rentrer dans les clous.

C'est lors de notre première réunion avec notre tutrice, Eva Ansermin, qu'on a clarifié tout ça. En discutant, on a compris comment réorienter le projet sans tout jeter. L'idée retenue a été la suivante : les points de référence du diagramme de Voronoï deviennent les réservoirs d'eau placés sur le terrain. À partir de là, le problème se reformule naturellement : pour chaque arroseur posé dans un champ, on cherche quel réservoir doit l'alimenter, c'est-à-dire le réservoir de la zone d'influence dans laquelle il se trouve.

On a gardé l'esprit du projet de départ, à savoir une gestion intelligente de l'irrigation, mais l'architecture a complètement changé. On a abandonné la partie capteurs et la notion d'administrateur ou de technicien pour se concentrer sur la modélisation géométrique, qui est le vrai cœur du sujet. Honnêtement, ce recentrage nous a un peu refroidi sur le moment, parce qu'on trouvait notre première version plus parlante. Mais avec le recul c'était le bon choix, parce que ça collait beaucoup mieux à ce qui était demandé et ça donnait un sens clair aux deux algorithmes.

3. Conception et choix techniques

Une fois la problématique fixée, on a défini les grandes briques de l'application. Tout tourne autour d'un terrain (la classe `Ground`) qui appartient à une personne (le propriétaire de l'exploitation). Ce terrain contient trois types d'éléments : des champs (`Field`), des réservoirs d'eau (`WaterTank`) et des arroseurs (`Sprinkler`).

Les réservoirs sont les sites du diagramme de Voronoï. Chaque arroseur est rattaché automatiquement au réservoir de la cellule dans laquelle il tombe, et s'il n'est dans aucune cellule on prend simplement le réservoir le plus proche. C'est ce lien arroseur vers réservoir qui constitue le résultat utile de l'application.

Pour le calcul, on a fait un choix important : on ne calcule pas les deux structures séparément. On calcule d'abord la triangulation de Delaunay, puis on en déduit le diagramme de Voronoï par dualité. Les deux structures sont en effet liées mathématiquement : chaque sommet d'une cellule de Voronoï correspond au centre du cercle circonscrit d'un triangle de Delaunay. Calculer un seul des deux et passer à l'autre nous évitait de faire deux fois le travail.

Au niveau de l'organisation du code, on a séparé les choses en plusieurs ensembles : le modèle métier (`Point`, `Field`, `WaterTank`, `Sprinkler`, `Ground`, `Person`), les algorithmes géométriques (`DelaunayTriangulation`, `DelaunayTriangle`, `VoronoiBuilder`, `VoronoiDiagram`, `VoronoiCell`), l'interface graphique en JavaFX, et la partie sauvegarde. Cette séparation nous a permis de travailler à plusieurs sans trop se marcher dessus.

Enfin, on fournit deux interfaces : une version en ligne de commande (CLI) et une version graphique JavaFX. La version CLI permet de tester tout le modèle de données indépendamment de l'interface graphique, et de montrer que toutes les fonctionnalités marchent même si certaines ne sont pas visibles à l'écran.

4. Organisation de l'équipe

Comme le projet mélangeait une grosse partie algorithmique et une grosse partie interface, on s'est réparti le travail en deux pôles dès le départ, juste après avoir mis à jour le diagramme de classes.

Tom, Paul et Oscar se sont occupés du modèle de données et des algorithmes, c'est-à-dire toute la logique "dure" du projet. De leur côté, Illya et Jilani ont pris en charge l'interface JavaFX. Comme JavaFX n'avait quasiment pas été vue en cours, Illya et Jilani ont d'abord passé du temps à se documenter et à poser les premières bases de l'interface (l'écran d'accueil, la structure générale) pendant que l'autre groupe avançait sur le code.

Une fois que le cœur algorithmique a été bien avancé, l'organisation a un peu bougé. Oscar s'est mis sur la version CLI, et Tom et Paul sont venus en renfort sur la partie JavaFX. Sur la fin, on s'est en réalité tous retrouvés sur l'interface graphique, parce que c'était la plus grosse partie qui restait à finir et qu'il fallait avancer vite. Paul a aussi géré, avant de rejoindre le groupe JavaFX, tout le mécanisme de sauvegarde et de chargement d'une ferme, qu'on a ensuite branché à l'interface.

Pour la communication, on s'est surtout organisés via un groupe WhatsApp où on se tenait au courant de ce que chacun avait push, et on utilisait GitHub pour le code, avec des commits plutôt réguliers, surtout sur les dernières semaines. On a aussi fait le point régulièrement avec notre tutrice : on a eu quatre réunions avec elle (le 22 mai, le 26 mai, le 5 juin et le 11 juin), qui nous ont à chaque fois aidés à mieux cadrer le projet et à corriger le tir sur certains points.

Le tableau ci-dessous résume les grandes phases du projet et leur enchaînement dans le temps.

Phase	Période	Membres impliqués
Conception / revue du sujet	17/05 - 26/05	Toute l'équipe
Modèle de données (classes)	26/05 - 01/06	Tom, Paul, Oscar
Debut JavaFX	26/05 - 03/06	Illya, Jilani
Algorithmes Delaunay / Voronoï	28/05 - 05/06	Tom, Oscar
Système sauvegarde	01/06 - 03/06	Paul
Version CLI	03/06 - 09/06	Oscar
Interface JavaFX	03/06 - 13/06	Illya, Jilani, Tom, Paul
Style, finitions, diagrammes UML, rapport	10/06 - 13/06	Toute l'équipe

Tableau 1 - Grandes phases du projet SmartFarm (avec également les réunions tutrice : 22/05, 26/05, 05/06, 11/06).

5. Réalisation technique

5.1 Le modèle de données

Tout part de la classe Point, qui représente simplement une position avec une coordonnée x et une coordonnée y. Elle contient une méthode pour calculer la distance entre deux points, qui nous sert un peu partout. Au début on avait une classe Coordinates en plus, mais elle faisait doublon avec Point, donc on l'a supprimée pour ne garder que Point. On est aussi passés des float aux double pour les coordonnées, parce que c'est plus simple à écrire et à manipuler.

La classe Field représente un champ, défini par un rectangle (ses bornes en x et en y), avec un nom et une surface. Les réservoirs (WaterTank) et les arroseurs (Sprinkler) héritent tous les deux de Point, puisqu'ils ont avant tout une position sur le terrain. Chacun possède en plus un identifiant unique généré automatiquement, ce qui permet par exemple de parler du "réservoir n°3" dans l'interface.

La classe Ground est le centre de tout : elle regroupe la liste des champs, des réservoirs et des arroseurs, connaît son propriétaire, et garde le diagramme de Voronoï une fois qu'il est calculé. Elle propose aussi quelques statistiques, comme le nombre d'arroseurs actifs, le nombre de réservoirs vides, ou encore le déséquilibre de charge entre réservoirs (la différence entre le réservoir qui alimente le plus d'arroseurs et celui qui en alimente le moins). C'est elle qui contient la méthode qui rattache un arroseur à son réservoir, en cherchant la cellule de Voronoï qui le contient.

5.2 Les algorithmes de Delaunay et Voronoï

C'est la partie la plus technique du projet, et celle qui nous a demandé le plus de réflexion. On détaille ici les grandes idées sans rentrer dans tout le code.

Pour la triangulation, on a implémenté l'algorithme de Bowyer-Watson. Le principe est d'insérer les points un par un. On part d'un grand triangle qui englobe tous les points (le super-triangle), puis pour chaque nouveau point on regarde quels triangles existants ne respectent plus la règle de Delaunay, on les retire, et on retriangule le trou ainsi créé en reliant le point aux bords. À la toute fin, on enlève les triangles qui touchent encore le super-triangle de départ.

La règle de Delaunay repose sur le cercle circonscrit : un triangle est valide si aucun autre point ne se trouve à l'intérieur de son cercle circonscrit. Pour calculer le centre de ce cercle (le circoncentre) à partir des trois sommets d'un triangle, on utilise la règle de Cramer. En notant le déterminant

$$d = 2 * (x1*(y2 - y3) + x2*(y3 - y1) + x3*(y1 - y2))$$

on en déduit les coordonnées du circoncentre par deux formules du même type (une pour x, une pour y), divisées par d. Si d est presque nul, c'est que les trois points sont quasiment alignés et ne forment pas un vrai triangle : on gère ce cas à part pour éviter une division qui partirait à l'infini. Une fois le centre connu, le rayon est juste la distance entre ce centre et n'importe lequel des trois sommets.

Le passage de Delaunay à Voronoï se fait ensuite par dualité. Pour chaque réservoir, on récupère tous les triangles qui le touchent, on prend les centres de leurs cercles circonscrits, et ces centres deviennent les sommets de la cellule de Voronoï du réservoir. On les trie par angle autour du réservoir pour former un polygone correct. Le schéma ci-dessous illustre ce lien entre les deux structures.

MapView avec les overlays Voronoï et Delaunay activés

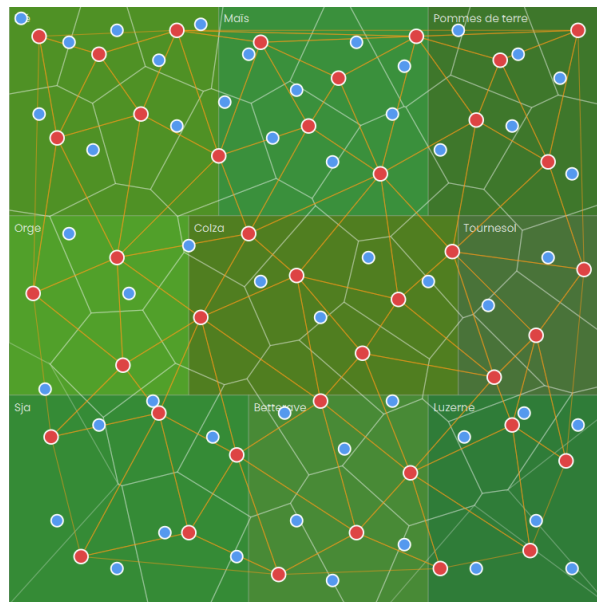


Image 2 - La vue carte avec le diagramme de Voronoï et la triangulation de Delaunay sur notre App.

Un problème classique est que les cellules situées au bord du terrain sont "ouvertes" : elles partiraient théoriquement à l'infini. Pour les fermer proprement dans les limites du champ, on a utilisé l'algorithme de découpage de Sutherland-Hodgman, qui découpe chaque polygone contre les quatre bords du terrain.

Chaque cellule sait aussi calculer son aire (avec la formule du lacet, qui donne l'aire d'un polygone à partir de ses sommets) et dire si un point se trouve à l'intérieur d'elle (avec la méthode du lancer de rayon : on compte combien de fois une demi-droite partant du point croise les bords du polygone ; si c'est impair, le point est dedans).

On a aussi vérifié que tout ça tenait la charge. Lors de nos tests, calculer la triangulation pour mille réservoirs prenait de l'ordre de la centaine de millisecondes, et le diagramme complet restait largement en dessous de la seconde. Ces mesures ont été faites en cours de développement et le code a continué d'évoluer derrière, mais elles nous ont rassurés sur le fait que la performance n'était pas un souci, même avec beaucoup de points.

Limite connue : il nous reste quelques cas particuliers de cellules dans les coins du terrain qui ne s'affichent pas parfaitement. Le découpage marche dans la grande majorité des cas mais ces quelques coins restent un point à améliorer, qu'on a préféré assumer plutôt que de bricoler une correction à la dernière minute.

5.3 L'interface graphique JavaFX

L'interface est la partie la plus visible du projet, et aussi celle qui a demandé le plus de travail à la fin. Quand on lance l'application, on arrive sur un écran d'accueil qui propose soit de créer une nouvelle ferme, soit de charger une ferme déjà sauvegardée.

Si on crée une nouvelle ferme, on passe par des formulaires pour renseigner les informations du propriétaire, puis pour ajouter les champs, les réservoirs et les arroseurs. Si on charge une sauvegarde, un écran liste les fermes enregistrées et on clique sur celle qu'on veut ouvrir.

Le cœur de l'interface est la vue carte. Elle affiche le terrain avec ses champs, les réservoirs en rouge et les arroseurs en bleu. On peut afficher ou masquer le diagramme de Voronoï et la triangulation de Delaunay par dessus la carte, zoomer et se déplacer dedans, ajouter un élément directement en cliquant à l'endroit voulu, et déplacer un réservoir ou un arroseur par glisser-déposer. Quand on clique sur un réservoir, les arroseurs qui lui sont rattachés se mettent en surbrillance, ce qui permet de visualiser d'un coup d'œil quelle pompe alimente quoi. Un panneau sur le côté affiche les informations de l'élément sélectionné et regroupe les boutons d'action (modifier, supprimer, etc.).

Pour le visuel, on a choisi un thème sombre dans les tons verts pour rester dans l'ambiance "agricole", avec une barre en haut commune à toutes les pages et une police un peu plus soignée que la police par défaut.

5.4 Aperçu de l'interface

Voici quelques captures de l'application telle qu'elle se présente en fonctionnement. La première montre l'écran d'accueil, et les suivantes illustrent la vue carte avec les différentes fonctionnalités activées.

Écran d'accueil de SmartFarm

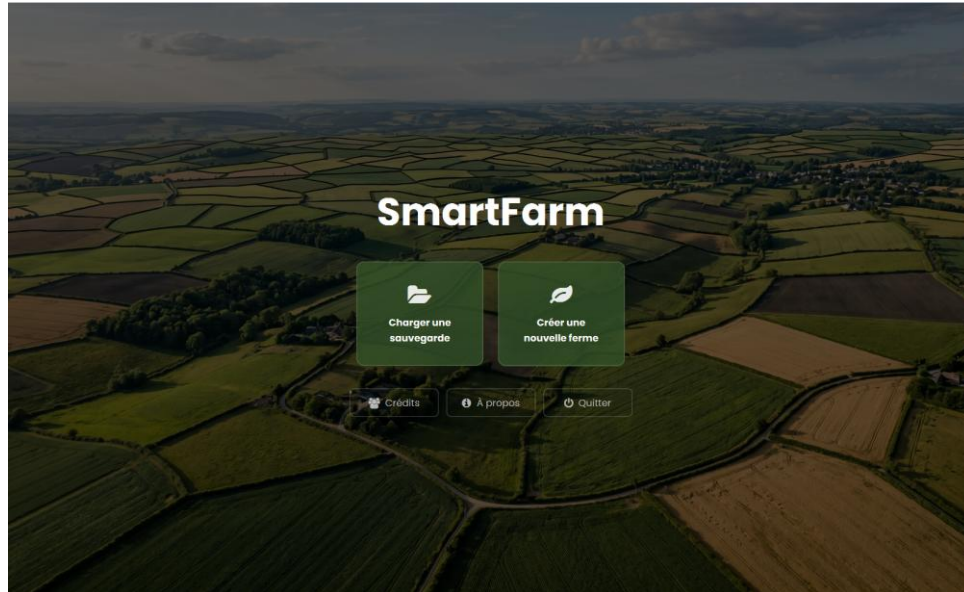


Image 3 - L'écran d'accueil

Vue carte (MapView) avec champs, réservoirs et arroseurs

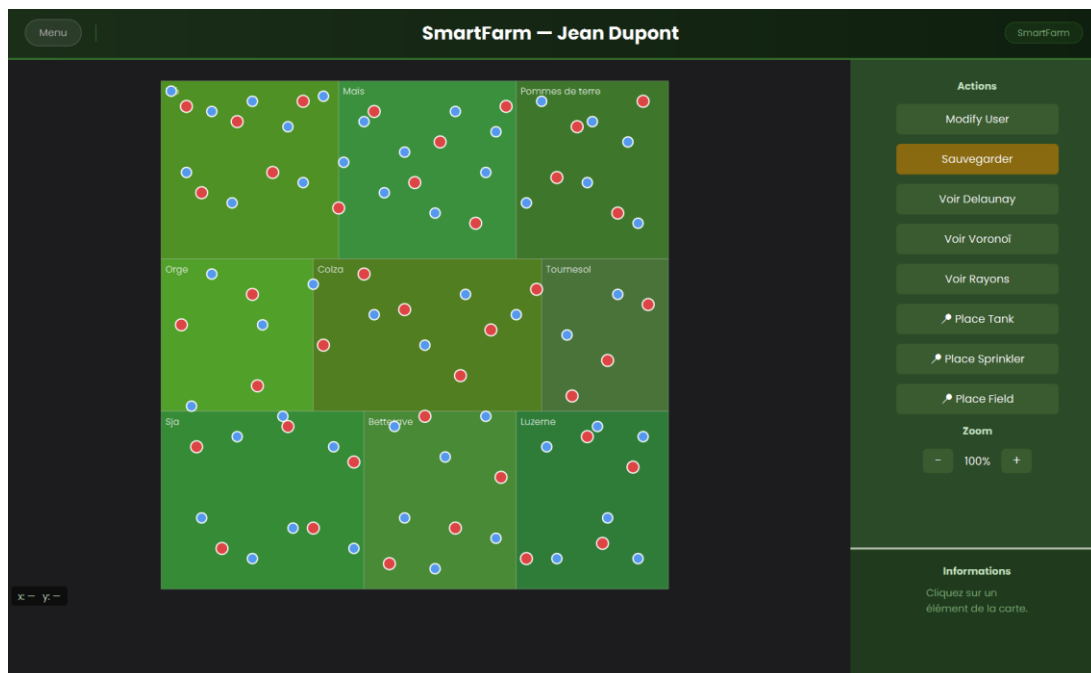


Image 4 - La vue carte avec les champs, les réservoirs (cercles rouges) et les arroseurs (cercles bleus).

5.5 La version en ligne de commande

En parallèle de l'interface graphique, on a développé une version en ligne de commande qui reprend toutes les fonctionnalités du modèle. Elle permet de gérer les réservoirs, les arroseurs et les champs, d'en ajouter en grand nombre d'un coup, d'inspecter le diagramme de Voronoï et la triangulation de Delaunay, et d'afficher les différentes statistiques. Comme demandé dans le cahier des charges, elle sert à montrer que toute la logique fonctionne indépendamment de l'affichage graphique.

5.6 La sauvegarde et le chargement

Pour sauvegarder une ferme, on enregistre dans un fichier le propriétaire, le terrain, puis la liste des champs, des réservoirs et des arroseurs, une ligne par élément, avec un point-virgule comme séparateur. Au chargement, on relit le fichier ligne par ligne pour reconstruire la ferme, et le diagramme de Voronoï est recalculé automatiquement à mesure qu'on rajoute les réservoirs. Le cahier des charges demandait un format de sauvegarde binaire. On est restés sur un **format texte**, qui est plus simple à lire et surtout beaucoup plus facile à déboguer quand quelque chose ne va pas. C'est un écart qu'on assume, fait en partie par manque de temps, et le passage à un format binaire fait partie des choses qu'on améliorerait avec un peu plus de marge.

6. Difficultés rencontrées

On regroupe ici les principaux obstacles qu'on a eu sur ce projet et la façon dont on les a gérés.

Le changement de sujet : La première difficulté a été de devoir revoir entièrement notre idée de départ après la première réunion. Comme on était partis sur une architecture à base de capteurs, on a dû repenser le projet et reprendre certaines choses, notamment le diagramme de classes. Par exemple, dans une première version on avait attribué le rayon au réservoir alors que c'est l'arroseur qui en a un : ce genre de petite erreur venait directement du fait qu'on avait une autre idée en tête au départ, et on les a corrigées en se relisant entre nous.

Le rythme de la période : On a mené ce projet en même temps que nos cours électifs, donc on avait cours certains jours en plein milieu de la période de développement, ce qui réduisait le temps qu'on pouvait y consacrer. Et la dernière semaine a été particulièrement chargée puisqu'il fallait à la fois finir le projet, préparer la soutenance et réviser les rattrapages. Caler les créneaux de chacun pour avancer ensemble n'a pas toujours été simple.

Comprendre Delaunay et Voronoï : Avant même de coder, il a fallu vraiment comprendre ces deux algorithmes : ce qu'ils font, comment ils sont liés, et de quelles briques on avait besoin pour les construire (le calcul du cercle circonscrit, la notion de dualité, le découpage des cellules sur les bords). C'est de la géométrie qu'on ne connaissait pas, et on a passé un certain temps au début à se l'approprier avant que ça se mette vraiment en place.

Les environnements de travail : On travaillait tous sous IntelliJ, mais certains d'entre nous le maîtrisaient déjà bien et d'autres beaucoup moins. Il y a donc eu un petit temps d'adaptation, et on a eu quelques accrocs sur des points de raccord entre nos configurations. Par exemple, un fichier de configuration personnel de l'IDE a été poussé par erreur sur le dépôt et a cassé les réglages d'un membre de l'équipe.

On a réglé ça en l'excluant du suivi Git. On a aussi eu, par moments, à ajuster des chemins de fichiers pour que l'application se comporte pareil d'une machine à l'autre.

7. Les diagrammes UML

Pour accompagner la conception, on a réalisé deux diagrammes UML qu'on a fait évoluer au fil du projet. Comme ils sont assez grands et détaillés, on a préféré les placer en annexe plutôt que de les écraser sur une page au format A4.

Le diagramme de classes (voir dans le dossier Rapport du projet) montre l'ensemble des classes et leurs relations. On y voit que la classe Ground regroupe les champs, les réservoirs et les arroseurs, que WaterTank et Sprinkler héritent de Point, et comment les classes liées à Voronoï et Delaunay (VoronoiDiagram, VoronoiCell, DelaunayTriangle) s'articulent avec les classes utilitaires qui font les calculs. Ce diagramme a pas mal changé entre le début et la fin, notamment au moment du recentrage du sujet.

Le diagramme de cas d'utilisation (voir dans le dossier Rapport du projet) décrit ce que l'utilisateur peut faire avec l'application : créer une nouvelle ferme ou en charger une, ajouter un champ, un réservoir ou un arroseur, activer ou désactiver un arroseur, consulter la carte (ce qui implique le calcul du diagramme de Voronoï et de la triangulation de Delaunay), zoomer, et quitter. Pris ensemble, la version graphique et la version en ligne de commande couvrent tous ces cas d'usage.

8. Conclusion

Au final, on a réussi à livrer une application qui fait ce qu'on attendait d'elle : un modèle de données complet, les deux algorithmes demandés (Delaunay et Voronoï), deux interfaces dont une graphique assez aboutie, et un système de sauvegarde. Le projet répond bien à la problématique qu'on s'était fixée, à savoir trouver pour chaque arroseur le réservoir qui doit l'alimenter en fonction des zones d'influence.

Sur le plan des compétences, ce qu'on retient surtout c'est la découverte de la géométrie algorithmique avec Voronoï et Delaunay, deux notions qu'on ne connaissait pas du tout avant et qui nous ont demandé un vrai effort de compréhension. On a aussi beaucoup progressé sur JavaFX, qu'on avait à peine vu en cours et qu'on a dû apprendre en grande partie par nous-mêmes. Le travail en équipe sur GitHub nous est aujourd'hui plus familier, vu qu'on l'utilise sur pas mal de projets, mais ça reste un point qu'on a consolidé ici.

Il reste évidemment des choses à améliorer. On aimerait corriger proprement l'affichage des cellules dans les coins du terrain, passer à un format de sauvegarde binaire comme le demandait le cahier des charges, enrichir les statistiques par zone, et proposer un ajout en masse de points directement depuis l'interface graphique.

Pour finir, ce projet a surtout été un vrai travail d'équipe. Chacun s'est investi sur sa partie et on a beaucoup avancé ensemble, en particulier sur la fin où on s'est tous mis sur l'interface pour boucler à temps. Les réunions régulières avec notre tutrice nous ont aidés à garder le cap, et on est plutôt contents du résultat.